

FaceID Security

Sistema de Reconhecimento Facial em Tempo Real

Documentação Técnica do Projeto

1. Objetivo

O projeto FaceID Security tem como objetivo desenvolver uma aplicação web capaz de cadastrar pessoas a partir de fotografias do rosto e, posteriormente, reconhecê-las automaticamente em duas modalidades: por meio de imagens enviadas pelo usuário e em tempo real através da webcam. Toda a inferência de visão computacional acontece dentro do próprio navegador, garantindo privacidade dos dados biométricos, enquanto a persistência das pessoas, descritores faciais e histórico de reconhecimentos é feita em um backend gerenciado em nuvem.

Como objetivos específicos, buscamos: (i) demonstrar a viabilidade do uso de redes neurais pré-treinadas executadas diretamente no cliente; (ii) integrar uma SPA moderna com banco de dados relacional e storage de objetos; e (iii) fornecer uma interface intuitiva, responsiva e com identidade visual coerente.

2. Tecnologias Utilizadas

2.1. Frontend

- **React 19** com **TypeScript** – construção da interface reativa e tipada.
- **TanStack Start + TanStack Router** – framework full stack com SSR e roteamento por arquivos.
- **Vite 7** – bundler/dev server de alta performance.
- **Tailwind CSS v4** – estilização baseada em tokens semânticos.
- **shadcn/ui** – biblioteca de componentes acessíveis (botões, inputs, cards).
- **Lucide React** – ícones SVG consistentes em toda a aplicação.

2.2. Visão Computacional

- **face-api.js** – wrapper sobre TensorFlow.js para detecção e reconhecimento facial.
- **TinyFaceDetector** – detector leve de bounding boxes de rostos.
- **FaceLandmark68Net** – extração de 68 pontos faciais.
- **FaceRecognitionNet** – geração de descritores de 128 dimensões para comparação.

2.3. Backend e Infraestrutura

- **Lovable Cloud** (PostgreSQL gerenciado, storage de objetos e autenticação).
- **RLS (Row Level Security)** – políticas de acesso configuradas por tabela.
- **Bun** – runtime JavaScript e gerenciador de pacotes.
- **Server Functions** do TanStack Start – lógica server-side encapsulada em RPCs tipadas.

3. Estrutura do Sistema

A aplicação segue o padrão de uma SPA com SSR, organizada em camadas bem definidas. O cliente carrega os modelos de rede neural a partir de `/public/models`, calcula os descritores faciais localmente e envia apenas vetores numéricos e arquivos de imagem para o backend.

3.1. Organização de Pastas

- **src/routes/** – rotas baseadas em arquivos (index, cadastro, upload, webcam, pessoas).
- **src/components/** – componentes reutilizáveis (AppNav, UI da shadcn).
- **src/hooks/** – hooks customizados (use-known-faces para cache dos descritores).
- **src/lib/face.ts** – lógica de carregamento de modelos, detecção e comparação.
- **src/integrations/supabase/** – clientes do banco (browser, server e admin).
- **supabase/migrations/** – schema SQL versionado.

3.2. Modelo de Dados

- **pessoas** – id, nome, cpf (único), data_cadastro.
- **imagens** – pessoa_id, caminho_imagem (storage), descriptor (JSONB de 128 floats).
- **reconhecimentos** – histórico com nome, confiança e origem (upload/webcam).
- **Bucket faces** – armazenamento público das fotografias.

3.3. Fluxo Resumido

1) O usuário cadastra uma pessoa enviando nome, CPF e foto. 2) O navegador calcula o descritor facial. 3) Foto vai para o bucket; descritor e metadados para o banco. 4) Em uma nova imagem ou frame de webcam, o descritor é extraído e comparado por distância euclidiana com os já armazenados. 5) Se a distância for menor que o limiar, a pessoa é reconhecida e o evento registrado no histórico.

4. Prints das Telas

A seguir são apresentadas capturas de tela das principais páginas da aplicação, ilustrando a navegação e a identidade visual do sistema.



Figura 1 – Página inicial com chamada principal e cards de funcionalidades.

A captura de tela mostra a tela de 'Cadastro de Pessoa'. No topo, a barra de navegação indica que o usuário está na seção 'Cadastro'. O título 'Cadastro de Pessoa' é seguido por uma instrução: 'Adicione nome, CPF e ao menos uma foto com rosto visível.' Abaixo, há uma barra de status que diz 'Carregando modelos de IA...'. O formulário contém campos para 'NOME' (com o valor 'João Silva') e 'CPF' (com o valor '000.000.000-00'). Abaixo desses campos, há uma seção para 'FOTOS FACIAIS (MÚLTIPLAS PERMITIDAS)' com uma área de upload que diz 'Clique para selecionar imagens'. Um botão 'Cadastrar pessoa' está na base do formulário.

Figura 2 – Formulário de cadastro de pessoa com upload de fotos.

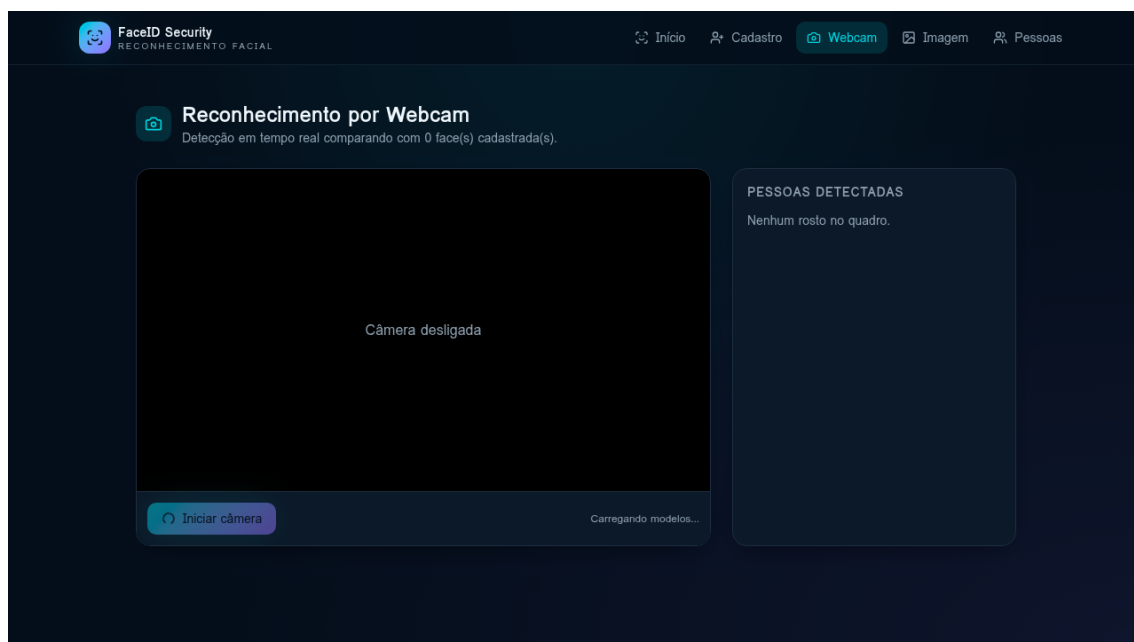


Figura 3 – Reconhecimento por webcam em tempo real.

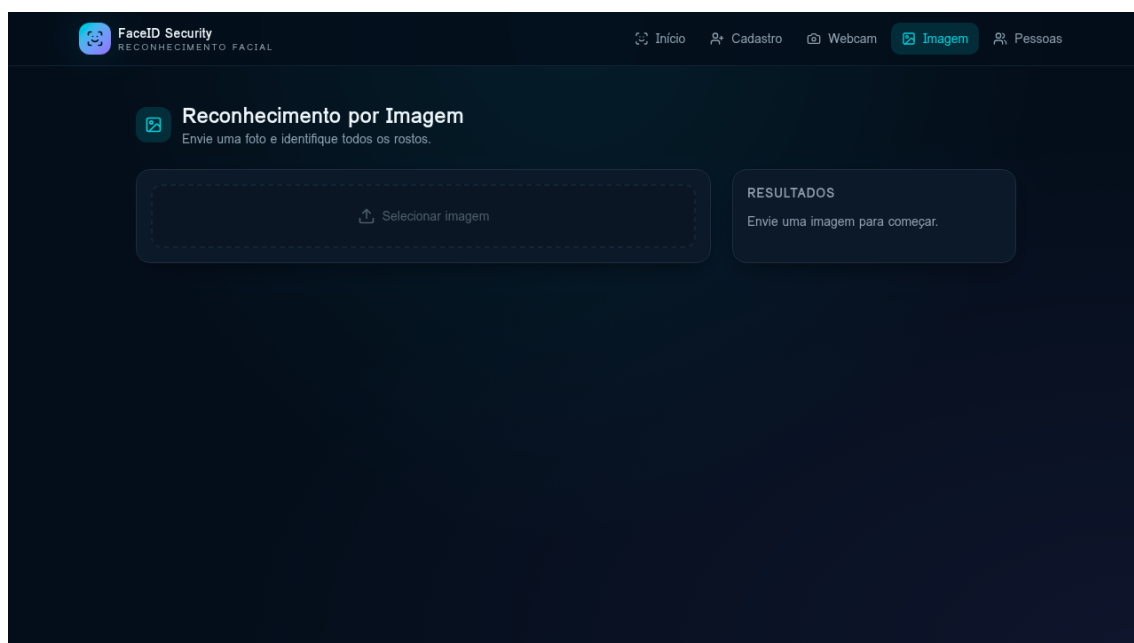


Figura 4 – Reconhecimento por imagem enviada.

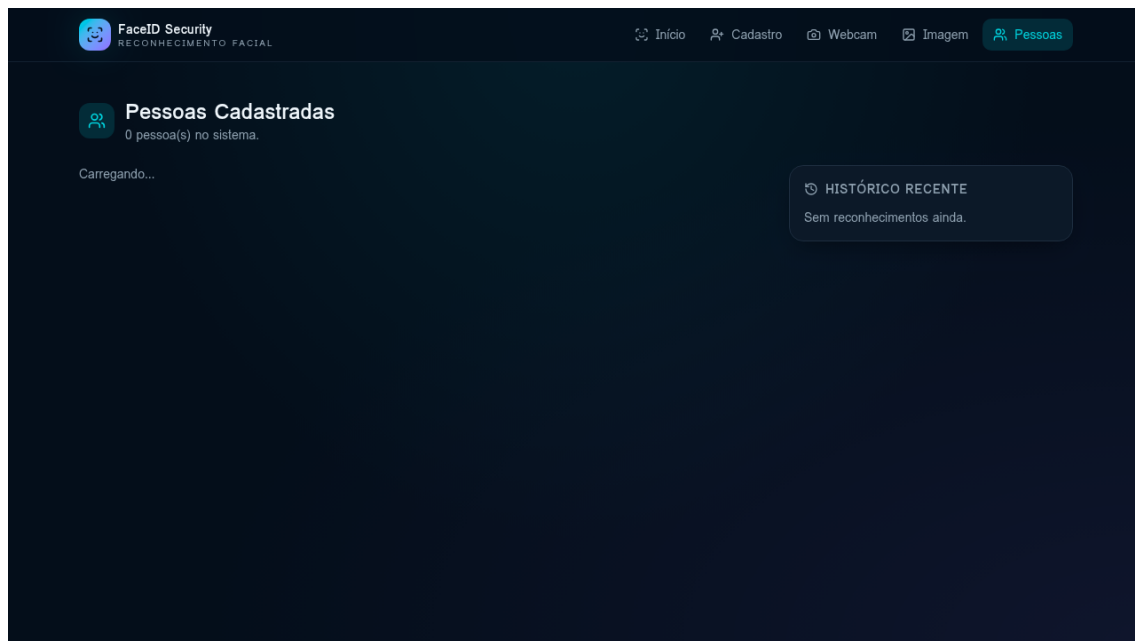


Figura 5 – Listagem de pessoas cadastradas e histórico recente.

5. Dificuldades Encontradas

5.1. Carregamento dos modelos no navegador

Os modelos do face-api.js somam vários megabytes e precisam ser baixados antes de qualquer detecção. No início, tentamos carregá-los sob demanda, o que gerava atrasos visíveis. A solução foi pré-carregar todos os modelos logo na entrada das telas que usam reconhecimento e exibir um indicador claro de carregamento.

5.2. Detecção em tempo real e desempenho

Rodar o detector a cada frame da webcam consumia muito processamento e travava o navegador em máquinas mais modestas. Foi necessário ajustar a frequência de detecção, reduzir a resolução de entrada e usar a versão TinyFaceDetector ao invés da SSD MobileNet, equilibrando precisão e fluidez.

5.3. Comparação confiável de descritores

Definir o limiar de distância euclidiana foi um desafio empírico. Valores muito baixos rejeitavam rostos legítimos; valores muito altos geravam falsos positivos. Após testes, fixamos o limiar em 0.6, que se mostrou um bom ponto de equilíbrio para fotos com iluminação razoável.

5.4. Permissões da câmera e HTTPS

Os navegadores só liberam o acesso à webcam em contextos seguros (HTTPS ou localhost). Durante o desenvolvimento local, foi preciso garantir que o ambiente respeitasse essa restrição, e em produção que o site estivesse publicado com certificado válido.

5.5. Persistência dos descritores no banco

Armazenar um array de 128 floats exigiu escolha cuidadosa do tipo de coluna. Optamos por JSONB no PostgreSQL pela flexibilidade de leitura e gravação direta via SDK, abrindo mão da indexação vetorial nativa, que ainda não era necessária para o volume esperado deste projeto.

5.6. Configuração inicial do ambiente

A integração entre TanStack Start, Vite, Tailwind v4 e shadcn/ui demandou ajustes finos na configuração de aliases, tokens semânticos de cor e ordem de imports. Padronizamos o uso exclusivo de variáveis CSS definidas em `src/styles.css`, evitando classes de cor diretas nos componentes.

5.7. SSR x código client-only

Como o face-api.js depende de APIs do navegador (canvas, getUserMedia), foi preciso isolar todo o código sensível em hooks executados apenas no lado do cliente, evitando erros do tipo *window is not defined* durante a renderização no servidor.